Module 5: Constraints

> Constraints are rules enforced on table columns to maintain data accuracy, consistency, and integrity. They prevent invalid data from entering the database and are one of the most important concepts in database design.

📖 Introduction

Imagine a company database where:

- Two employees have the same EmployeeID
- A student's age is entered as -5
- An order references a customer that doesn't exist

These issues create data integrity problems.

Constraints solve these problems by enforcing rules at the database level.

🎯 Learning Objectives

After completing this module, you will be able to:

✅ Understand why constraints are important

✅ Apply different types of constraints

✅ Maintain data integrity

✅ Prevent duplicate and invalid data

✅ Create relationships between tables

✅ Design production-ready databases

📚 Table of Contents

1️⃣ What are Constraints?

Constraints are rules applied to table columns that restrict the type of data that can be inserted, updated, or deleted.

Example

Without Constraint:

```
INSERT INTO Students
VALUES
(101,'Kunal',-10);
```

Age = -10 is invalid.

With Constraint:

```
CHECK (Age >= 0)
```

Database rejects invalid data.

2️⃣ Why Constraints Matter

Constraints help ensure:

Data Accuracy

Only valid data enters the database.

Data Consistency

All records follow the same rules.

Data Integrity

Relationships remain valid.

Reduced Errors

Many mistakes are caught automatically.

Real-World Example

Bank Database:

Without constraints:

```
Account Balance = -100000
```

Possible.

With constraints:

```
CHECK (Balance >= 0)
```

Invalid values rejected.

3️⃣ Types of Constraints

The most commonly used SQL constraints are:

Constraint	Purpose
NOT NULL	Value required
UNIQUE	No duplicate values
PRIMARY KEY	Unique + Not Null
FOREIGN KEY	Creates relationships
CHECK	Validates values
DEFAULT	Provides default value

4️⃣ NOT NULL Constraint

Ensures a column cannot contain NULL values.

Without NOT NULL

```
CREATE TABLE Students
(
    StudentID INT,
    StudentName VARCHAR(100)
);
```

Allowed:

```
INSERT INTO Students
VALUES
(101,NULL);
```

With NOT NULL

```
CREATE TABLE Students
(
    StudentID INT,
    StudentName VARCHAR(100) NOT NULL
);
```

Now:

```
INSERT INTO Students
VALUES
(101,NULL);
```

Result:

```
ERROR
Cannot insert NULL value.
```

Real-World Usage

Use NOT NULL for:

- Names
- Email Addresses
- Order Dates
- Product Names

5️⃣ UNIQUE Constraint

Ensures all values in a column are different.

Example

Email addresses should be unique.

Create Table

```
CREATE TABLE Employees
(
    EmployeeID INT,
    Email VARCHAR(100) UNIQUE
);
```

Valid

```
a@gmail.com
b@gmail.com
c@gmail.com
```

Invalid

```
a@gmail.com
a@gmail.com
```

Database rejects duplicate value.

Multiple UNIQUE Columns

```sql
CREATE TABLE Employees
(
    EmployeeID INT UNIQUE,
    Email VARCHAR(100) UNIQUE
);
```

UNIQUE vs PRIMARY KEY

Feature	UNIQUE	PRIMARY KEY
Duplicates Allowed	❌	❌
NULL Allowed	✅ (One NULL in SQL Server)	❌
Multiple Per Table	✅	❌

6️⃣ PRIMARY KEY Constraint

Uniquely identifies every row.

Rules

A Primary Key:

✅ Must be unique

✅ Cannot be NULL

✅ One per table

Example

```sql
CREATE TABLE Students
(
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(100)
);
```

Valid

```
101
102
103
```

Invalid

Duplicate:

```
101
101
```

NULL:

```
NULL
```

Why Primary Keys Matter

Every table should have a unique identifier.

Examples:

Table	Primary Key
Students	StudentID
Employees	EmployeeID
Orders	OrderID
Products	ProductID

7️ FOREIGN KEY Constraint

Creates relationships between tables.

Parent Table

```sql
CREATE TABLE Departments
(
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(100)
);
```

Child Table

```sql
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,

    EmployeeName VARCHAR(100),

    DepartmentID INT,

    FOREIGN KEY (DepartmentID)
    REFERENCES Departments(DepartmentID)
);
```

Relationship

```
Departments
│
├── DepartmentID (PK)
│
▼

Employees
│
└── DepartmentID (FK)
```

Valid Insert

Departments:

```
1 → HR
2 → IT
```

Employee:

```
DepartmentID = 1
```

Allowed.

Invalid Insert

```
DepartmentID = 10
```

Rejected.

Reason:

Department does not exist.

Benefits of Foreign Keys

✅ Maintains referential integrity

✅ Prevents orphan records

✅ Creates reliable relationships

---

8 CHECK Constraint

Validates values before insertion.

Example

Age cannot be negative.

```
CREATE TABLE Students
(
    StudentID INT PRIMARY KEY,
```

```
    Age INT CHECK (Age >= 0)
);
```

Valid

```
18
20
25
```

Invalid

```
-5
```

Rejected.

Multiple Conditions

```
CHECK
(
    Age >= 18
    AND
    Age <= 60
)
```

Example

```
CREATE TABLE Employees
(
    Salary DECIMAL(10,2)
    CHECK (Salary > 0)
);
```

Common CHECK Examples

```
CHECK (Marks BETWEEN 0 AND 100)

CHECK (Age >= 18)

CHECK (Salary > 0)

CHECK (Gender IN ('M','F'))
```

---

9 DEFAULT Constraint

Automatically assigns a value if none is provided.

Example

```
CREATE TABLE Employees
(
    Country VARCHAR(50)
    DEFAULT 'India'
);
```

Insert

```
INSERT INTO Employees
VALUES ('Kunal');
```

Stored:

```
India
```

automatically.

Example

```
CREATE TABLE Orders
(
    OrderDate DATE
```

```
    DEFAULT GETDATE()
);
```

Result

Current date inserted automatically.

🔟 Composite Constraints

Constraints involving multiple columns.

Composite Primary Key

```
CREATE TABLE StudentCourses
(
    StudentID INT,

    CourseID INT,

    PRIMARY KEY
    (
        StudentID,
        CourseID
    )
);
```

Why?

Prevents duplicate enrollments.

Valid

```
1,101
1,102
```

Invalid

```
1,101
1,101
```

1️⃣1️⃣ Naming Constraints

Production databases often use named constraints.

Example

```
CREATE TABLE Employees
(
    EmployeeID INT,

    CONSTRAINT PK_Employees
    PRIMARY KEY (EmployeeID)
);
```

Named CHECK Constraint

```
CREATE TABLE Employees
(
    Salary DECIMAL(10,2),

    CONSTRAINT CHK_Salary
    CHECK (Salary > 0)
);
```

Advantages

✅ Easier debugging

✅ Easier maintenance

✅ Professional design

1️⃣2️⃣ Best Practices

Always Use Primary Keys

Every table should have one.

Apply NOT NULL Carefully

Required fields should never be NULL.

Use CHECK Constraints

Validate business rules.

Use FOREIGN KEYS

Maintain relationships.

Use Meaningful Constraint Names

Example:

```
PK_Employees
FK_Orders_Customers
CHK_Salary
```

Avoid Excessive Constraints

Too many constraints may reduce flexibility.

1️⃣3️⃣ Common Mistakes

No Primary Key

Bad:

```
CREATE TABLE Employees
(
    Name VARCHAR(100)
);
```

Wrong CHECK Logic

Bad:

```
CHECK (Salary < 0)
```

Missing Foreign Key

Results in orphan records.

Using VARCHAR for IDs

Bad:

```
EmployeeID VARCHAR(100)
```

Better:

```
EmployeeID INT
```

Real-World Example

Employee Management System

```
CREATE TABLE Employees
(
    EmployeeID INT
    CONSTRAINT PK_Employees
    PRIMARY KEY,

    EmployeeName NVARCHAR(100)
    NOT NULL,

    Email VARCHAR(150)
    CONSTRAINT UQ_Email
    UNIQUE,

    Salary DECIMAL(10,2)
    CONSTRAINT CHK_Salary
    CHECK (Salary > 0),
```

```
    Country VARCHAR(50)
    CONSTRAINT DF_Country
    DEFAULT 'India'
);
```

📝 Summary

In this module, you learned:

✅ NOT NULL

✅ UNIQUE

✅ PRIMARY KEY

✅ FOREIGN KEY

✅ CHECK

✅ DEFAULT

✅ Composite Keys

✅ Named Constraints

✅ Data Integrity Concepts

✅ Production Database Best Practices

🧠 Practice Questions

Theory

1. What is a constraint?
2. Why are constraints important?
3. Difference between UNIQUE and PRIMARY KEY?
4. What is a FOREIGN KEY?
5. What is referential integrity?
6. What is a CHECK constraint?
7. What is a DEFAULT constraint?
8. Why use NOT NULL?
9. What is a composite primary key?
10. Why name constraints?

Practical Exercises

Task 1

Create:

```
Students
```

Requirements:

- StudentID Primary Key
- StudentName NOT NULL
- Email UNIQUE
- Age CHECK (Age >= 16)

Task 2

Create:

```
Departments
Employees
```

Add:

- Primary Key
- Foreign Key

Task 3

Create Product Table

Requirements:

- ProductID Primary Key
- ProductName NOT NULL
- Price > 0
- StockQuantity Default 0

Challenge Project

Design a Library Management System.

Tables:

```
Books
Members
```

```
BorrowRecords
```

Apply:

- Primary Keys
- Foreign Keys
- CHECK Constraints
- DEFAULT Values
- Named Constraints

🚀 Next Module

➞ Module 6: INSERT Statement

Topics Covered:

- INSERT INTO
- Inserting Single Row
- Inserting Multiple Rows
- Inserting Specific Columns
- INSERT INTO SELECT
- Handling NULL Values
- Common Errors & Best Practices